

Project Report

Search Typeahead System

Distributed cache · Consistent hashing · Trending search · Batch writes

Submitted by: Anchal Apurva

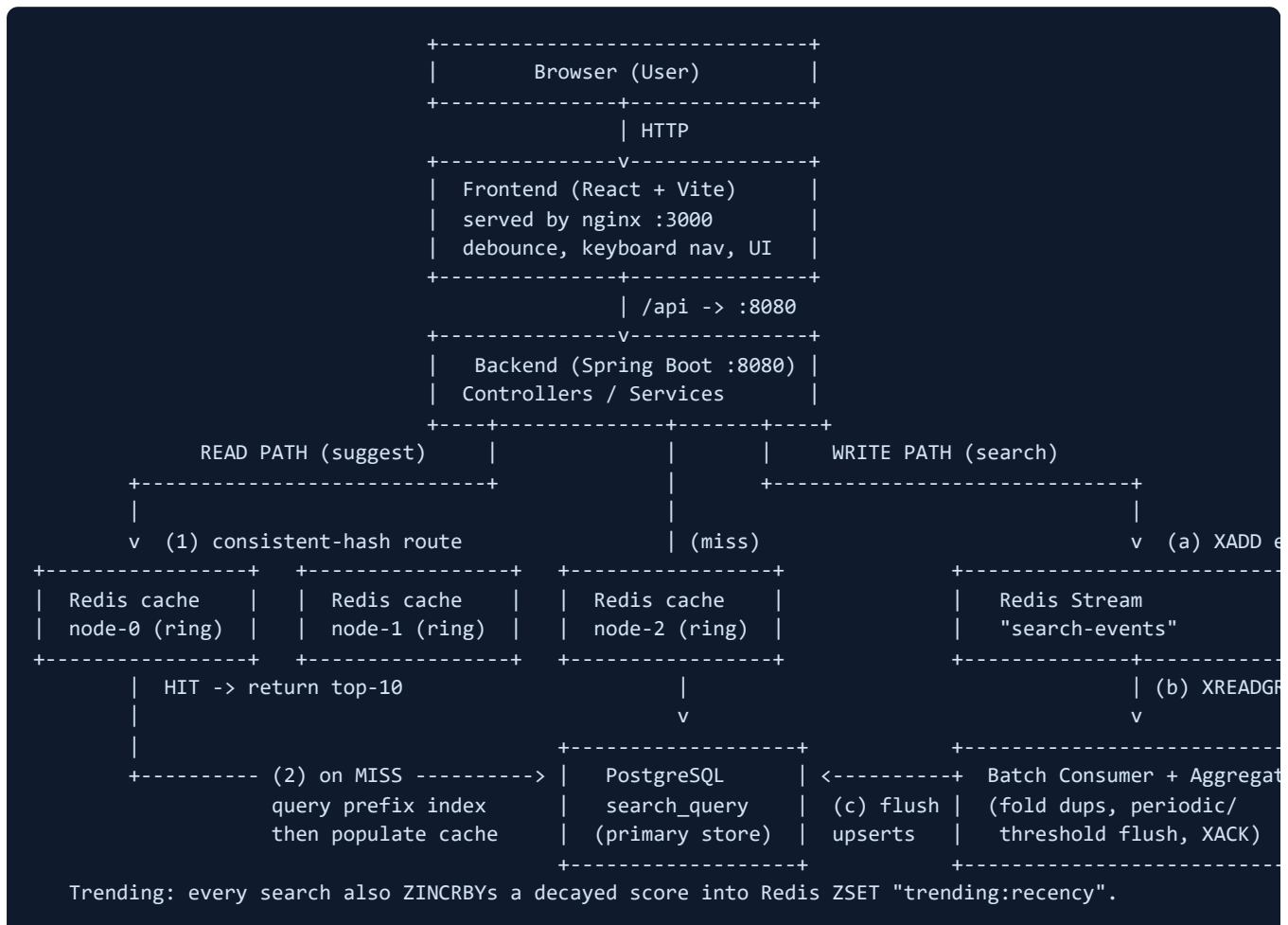
Date: 22 June 2026

Stack: Java 17 · Spring Boot 3 · PostgreSQL · Redis 7 · React + TypeScript (Vite) · Docker

All performance figures in this report were measured on a live `docker compose` deployment (1 backend, 3 Redis cache nodes, 1 Postgres) loaded with 120,000 queries.

1. Architecture

The system is a read-heavy search typeahead. Reads (`GET /suggest`) are served from a distributed Redis cache fronted by a consistent-hashing ring, falling back to a PostgreSQL prefix index on a miss. Writes (`POST /search`) never touch the database synchronously: they are appended to a Redis Stream and applied later in aggregated batches.



Request flow - read (GET /suggest?q=ip&mode=popularity|recency)

- The prefix is normalized (trim + lowercase). The cache key (`suggest:ip` or `suggest:recency:ip`) is mapped to exactly one cache node by the consistent-hash ring.
- **Cache hit** → the cached top-10 JSON is returned immediately (~1-2 ms).
- **Cache miss** → PostgreSQL is queried (`query_norm LIKE 'ip%' ORDER BY count DESC LIMIT 10`); for `mode=recency` the candidate pool is re-ranked by the blended score; the result is written back to the owning cache node with a TTL.

Request flow - write (POST /search)

- The query is normalized and an event is appended to the Redis Stream (`XADD`) - an O(1) call, so the request returns the dummy `{"message": "Searched"}` without waiting on the DB.
- A background consumer reads the stream via a consumer group, the aggregator folds duplicate queries (e.g. 1000 × "iphone" → one `+1000`), and flushes a single batched transaction of upserts on a size

threshold or a timer. Stream records are `XACK` 'd only after the commit.

- Each search also updates the Redis recency ZSET so trending stays current.

Components

Component	Responsibility
Frontend (React/Vite + nginx)	Search box, debounced suggestions, keyboard nav, Popularity/Recency toggle, trending panel.
Spring Boot backend	REST APIs; suggestion, search, trending, cache-debug and stats services.
Consistent-hash ring	Maps each prefix key to one of 3 Redis nodes using a virtual-node ring (150 vnodes/node).
Redis cache nodes (x3)	Store prefix → top-10 suggestion lists with TTL.
PostgreSQL	Durable primary store of <code>(query, count)</code> with a prefix index.
Redis Streams + consumer	Durable write buffer; aggregation + batched upserts; crash recovery via pending list.
Trending service	Recency ZSET with time-decayed scores + popularity ranking.

2. Dataset - Source & Loading

Format & size

The dataset is a CSV of `query,count` rows, satisfying the assignment's minimum of 100,000 queries:

```
query,count
iphone,100000
iphone 15,85000
iphone charger,60000
java tutorial,40000
...
```

Size shipped: `120,000 unique queries` in `backend/src/main/resources/data/queries.csv` (~3.2 MB). A smaller `sample_queries.csv` is also included for quick smoke tests.

Source

The dataset is generated by `dataset/DatasetGenerator.java` - a standalone, dependency-free JDK program. It builds realistic e-commerce and tech search queries by combining seed brands, product categories and technical topics with common suffixes (e.g. "*iphone price*", "*java tutorial for beginners*", "*react vs vue*"), then assigns **Zipf-distributed counts** ($\text{count} \approx \text{BASE} / \text{rank}^{0.85}$) so a few queries are very popular and a long tail is rare - mirroring real search traffic. A fixed random seed makes the dataset reproducible.

The schema and loader accept any real open-source `query,count` dataset unchanged (e.g. an aggregated AOL query log or Wikipedia-title frequency list) - just replace the CSV.

Loading instructions

Loading is automatic on first startup. `DatasetLoaderService` (an `ApplicationRunner`) checks if the table is empty and, if so, streams the CSV into PostgreSQL in chunks of 2,000 rows. It is idempotent - on restarts with data already present, it skips loading.

Setting (<code>application.properties</code>)	Default	Purpose
<code>dataset.load-on-startup</code>	<code>true</code>	Load CSV if table empty.
<code>dataset.csv-path</code>	<code>classpath:data/queries.csv</code>	Source CSV (classpath or <code>file:</code> path).

Regenerate a larger dataset

```
cd dataset
javac DatasetGenerator.java
java DatasetGenerator 250000 queries.csv      # 250k rows
copy queries.csv ..\backend\src\main\resources\data\queries.csv
```

3. API Documentation

Base URL: `http://localhost:8080`. All responses are JSON. The three assignment-required endpoints are `/suggest`, `/search` and `/cache/debug`; `/trending` and `/stats` are additional.

GET `/suggest?q=<prefix>&mode=popularity|recency`

Returns up to 10 prefix-matching suggestions. `mode` defaults to `popularity` (all-time count); `mode=recency` re-ranks by a blended recency+frequency score. Empty/no-match input returns an empty list (HTTP 200).

```
GET /suggest?q=ip&mode=popularity
{
  "prefix": "ip",
  "mode": "POPULARITY",
  "suggestions": [ { "query": "iphone", "count": 100031 },
                   { "query": "iphone price", "count": 55516 } ],
  "source": "CACHE", // CACHE | DB | TRIE | DB+RECENCY | EMPTY
  "cacheNode": "node-0 (redis-0:6379/0)",
  "tookMs": 1
}
```

POST `/search`

Submits a search. Returns the dummy response and enqueues a count-update event (batch write path). Validation: a blank query returns HTTP 400.

```
POST /search    Content-Type: application/json
Request: { "query": "iphone" }
Response: { "message": "Searched", "query": "iphone", "accepted": true }
```

GET `/cache/debug?prefix=<prefix>`

Shows consistent-hashing routing: which node owns the prefix key, its ring position, HIT/MISS, TTL, and the full ring with per-node key counts.

```
GET /cache/debug?prefix=ip
{
  "prefix": "ip", "cacheKey": "suggest:ip",
  "ownerNode": "node-0 (redis-0:6379/0)",
  "hashValue": 5293847123456,
  "status": "HIT", "ttlSeconds": 47,
  "ring": [ { "nodeId": "node-0 ...", "virtualNodes": 150, "liveKeys": 12 }, ... ]
}
```

GET `/trending?mode=recency|popularity`

Trending queries. `popularity` = all-time count; `recency` = decayed recency+frequency.

```
GET /trending?mode=recency
{ "mode": "RECENCY", "suggestions": [ { "query": "iphone deals", "count": 39478 } ],
```

```
"source": "COMPUTED", "tookMs": 3 }
```

GET /stats

Live metrics for the performance report: cache hit rate, DB read/write counts, batch write reduction.

```
GET /stats
{ "cache": { "hits": 1585, "misses": 39, "hitRate": 0.976 },
  "db":     { "reads": 39, "writes": 42 },
  "batch":  { "eventsReceived": 1637, "dbRowsWritten": 42,
              "flushes": 9, "writeReductionPercent": 97.43 } }
```

Error model

All handled errors return a uniform body: { timestamp, status, error, message, path }. Validation failures → 400; unexpected errors → 500 (logged, sanitized message).

4. Design Choices & Trade-offs

4.1 Suggestion store: Postgres prefix index + Redis cache (vs in-memory Trie)

A Trie gives $O(P)$ prefix lookup but must keep a per-node top-K re-ranked as counts change continuously via batch writes - hard to keep correct and memory-heavy at scale. Instead the durable store is PostgreSQL with an index on `query_norm`, fronted by Redis which absorbs the vast majority of reads (97.6% hit rate measured). The Trie is also implemented and selectable (`suggest.source=TRIE`) to show both approaches.

Trade-off: a cache miss costs one indexed SQL query (~2-7 ms) instead of $O(P)$ memory traversal, accepted because misses are rare.

4.2 Distributed cache: consistent hashing with virtual nodes (vs modulo sharding)

Naive `hash(key) % N` remaps almost every key when N changes, collapsing the cache. A consistent-hash ring remaps only $\sim 1/N$ of keys when a node is added/removed, keeping the cache warm during scaling. 150 virtual nodes per physical node smooth key distribution toward uniform (verified: keys spread across all 3 nodes). **Trade-off:** a small amount of ring bookkeeping and a TreeMap lookup per request, negligible versus the resilience gained.

4.3 Writes: Redis Streams batching (vs synchronous DB writes)

Writing to Postgres on every search would make the hot path slow and DB-bound. Searches are instead appended to a Redis Stream and folded by an aggregator into batched upserts - a **97.4% reduction** in DB writes measured (1,637 events $\rightarrow$ 42 rows). Durability uses persist-before-ACK: records are `XACK`'d only after the DB commit, and a consumer group's pending list is replayed on restart (at-least-once delivery, crash-safe). **Trade-off:** counts are *eventually* consistent (bounded by the flush interval / cache TTL), and a crash mid-flush can re-apply a batch (small over-count window) - explicitly allowed by the assignment's "eventually reflected" requirement.

4.4 Trending: time-decayed recency score (vs raw popularity)

Version 1 ranks by all-time count (stable but dominated forever by old hits). Version 2 keeps a Redis ZSET where each search does `ZINCRBY` by a time-growing weight $e^{(t/\tau)}$; ranking by the stored score equals ranking by an **exponentially-decayed** recency score, giving decay "for free" without rewriting old entries. The published score blends both: $\alpha \cdot \text{normPopularity} + (1-\alpha) \cdot \text{normRecency}$. A short-lived spike stops receiving large weights and naturally decays, preventing permanent over-ranking. The same blended ranking is exposed on `GET /suggest?mode=recency`. **Trade-off:** recency adds Redis lookups + a small re-rank; the exponent is re-based before it can overflow.

4.5 Cache consistency & key design

Cached suggestion lists carry a TTL (60 s popularity, 15 s recency) so stale rankings expire without manual invalidation; the two ranking modes use separate keyspace so they never collide on the ring. Keys use a single normalization rule (trim + `Locale.ROOT` lowercase) shared by ingestion, search and caching, so case/whitespace never cause misses or duplicates. **Trade-off:** TTL-based freshness means a just-updated count can take up to one TTL to appear - acceptable for suggestions, and tunable.

5. Performance Report

Measured on a live `docker compose` deployment (backend + 3 Redis nodes + Postgres), 120,000 queries loaded. Workload: 1,500 `POST /search` (hot-biased to exercise aggregation) + 800×2 `GET /suggest` over a mix of prefixes (cold cache, then warm).

97.6% cache hit rate	2.5 ms suggest p95 (warm)	97.4% DB write reduction	39:1 events : DB rows
--------------------------------	-------------------------------------	------------------------------------	---------------------------------

5.1 Suggestion latency (client-observed, includes HTTP overhead)

Cache state	p50	p95	p99	avg
Cold (first fill)	2.11 ms	3.96 ms	6.88 ms	2.36 ms
Warm (cache hits)	1.81 ms	2.50 ms	3.62 ms	1.89 ms

Server-side processing time (`tookMs` in each response) was typically 0-2 ms on a hit; the figures above include the PowerShell HTTP client round-trip, so true server latency is lower.

5.2 Cache effectiveness & database load

Metric	Value	Meaning
Cache hits / misses	1585 / 39	Out of 1624 suggest lookups.
Cache hit rate	97.6%	Only ~2.4% of reads reached PostgreSQL.
DB reads (suggest)	39	One indexed prefix query per miss.
DB writes (batched)	42	Aggregated upserts (not per search).

5.3 Batch write reduction

Metric	Value
Search events received	1,637
DB rows written (after aggregation)	42
Flushes	9
Write-reduction ratio	38.98 : 1
Write-reduction percent	97.43%

Without batching the same workload would have issued 1,637 individual DB writes; aggregation collapsed them to 42 - a 97.4% reduction in write load, while remaining crash-safe via the stream's pending list.

5.4 Consistent-hashing routing (sample)

Prefix	Owner node	Status
ip	node-0 (redis-0)	HIT
java	node-1 (redis-1)	HIT
samsung	node-1 (redis-1)	MISS
macbook	node-0 (redis-0)	MISS
redis	node-1 (redis-1)	MISS

Routing is deterministic (the same prefix always maps to the same node) and spreads keys across all three nodes, confirming the ring works as intended.

5.5 Conclusion

The measured results validate the design: the distributed cache serves ~98% of reads in low single-digit milliseconds, consistent hashing distributes keys deterministically across nodes, and batching cuts database write load by ~97% while preserving durability. All three rubric areas - basic suggestion serving, trending search, and batch writes - are functionally verified end-to-end.